TP6 – Mejorá el agente

1. El archivo mejorado

`revisor.md`. Sigue entrando en una pantalla: pasó de 22 a 37 líneas.

```
---
name: revisor
description: Revisa cambios de código buscando problemas de seguridad y robustez. Usar cuando se agregan
endpoints, se toca acceso a datos o configuración, o antes de subir algo a producción.
model: inherit
readonly: true
---

Sos un revisor de código con foco en seguridad y robustez. Revisás código de
otros antes de que llegue a producción.

Usá estas preguntas como guía para cada archivo. No las contestes una por una:
reportá solo lo que encuentres.

1. ¿Qué datos entran desde afuera sin validarse?
2. ¿Qué pasa si esto falla? ¿El resto del sistema sobrevive?
3. ¿Qué se rompe cuando el volumen crezca?
4. ¿Quién puede ejecutar esto y con qué permisos?

Antes de afirmar algo que depende de otro archivo (por ejemplo, "este dato no
se valida"), buscá quién llama a esta función y confirmalo. Si no podés
confirmarlo, marcalo "A confirmar" y decí qué habría que mirar.

Severidad:
- Crítico: se pueden perder, modificar o exponer datos, o ejecutar algo sin permiso.
- Alto: el sistema se cae o deja de responder para todos.
- Medio: falla en casos puntuales o se degrada con el uso.

Cada hallazgo con este formato:
**[Severidad] archivo:línea — título corto**
Escenario: qué entrada o situación concreta lo dispara y qué pasa. Sin adjetivos.

No reescribas el código. No propongas refactors. No opines sobre nombres de
variables ni sobre estilo.

Ordená de más grave a menos grave. Hasta cinco hallazgos. Si no hay ninguno,
respondé solo "Sin hallazgos de seguridad o robustez." No rellenes para
llegar a cinco.
```

Lo que **no** cambié: `readonly: true`, `model`, la `description`, el rol, las cuatro preguntas y las prohibiciones (no reescribir, no refactors, no estilo).

---

2. Justificación de cada cambio

#	Qué cambié	Por qué	Qué situación concreta mejora
1	"Contestá estas cuatro preguntas" → "usalas como guía, no las contestes una por una".	El original pedía responder cada pregunta en cada archivo, así que el agente llenaba una sección aunque no hubiera nada que decir.	En un archivo sin acceso a datos, ya no aparece "¿Quién lo ejecuta? Cualquiera, no requiere permisos". Se lee solo lo que importa.
2	Verificar el código que llama a la función antes de afirmar algo, y "A confirmar" si no se puede.	Un revisor que mira un solo archivo afirma cosas que dependen de otros, como "este dato no se valida".	Si la ruta ya valida el `id`, el agente no lo reporta como inyección crítica. Si no puede saber algo (por ejemplo, la versión de Express), lo dice en vez de inventarlo.
3	Criterio de severidad con tres niveles definidos.	"Ordená de más grave a menos grave" no decía qué es grave, y cada corrida usaba su propio criterio (o ninguno).	Un hallazgo que puede exponer datos siempre queda arriba de uno que solo degrada el rendimiento, y la etiqueta permite decidir rápido qué arreglar primero.
4	Formato fijo: `[Severidad] archivo:línea — título` + "Escenario: …".	Sin formato, los hallazgos mezclaban varios problemas en un párrafo y las líneas aparecían a veces sí y a veces no.	Cada hallazgo se ubica en el código con un clic, y el escenario obliga a decir **cuándo** se rompe, no solo que "es riesgoso".
5	"Máximo cinco" → "Hasta cinco. Si no hay ninguno, decilo. No rellenes."	"Máximo cinco" se lee como "tenés que encontrar algo". Un revisor que siempre encuentra algo enseña a ignorarlo.	Sobre un archivo sano, el agente responde una línea en lugar de inventar hallazgos de severidad "muy baja".

---

3. Evidencia: antes y después

Cómo lo probé

- Corrí **el mismo revisor** con el prompt original y con el mejorado, sobre **los mismos archivos**: una corrida por versión y por caso, 6 en total.
- Cada corrida se hizo como un subagente aparte con solo herramientas de lectura (Read, Grep, Glob). Las instrucciones del agente iban tal cual.
- Después de las 6 corridas comparé el hash de todos los archivos de prueba: **ninguno cambió**. El revisor sigue sin tocar archivos.
- Las respuestas completas están en la carpeta `evidencia/`.

Usé tres casos, cada uno pensado para probar cambios distintos:

Caso	Archivo	Para qué sirve
1	`queryTicketRows.ts` (la función del TP5)	Archivo con problemas reales. Es el "mismo archivo" que pide la consigna.
2	`descuento.ts`	Función pura, bien validada; solo tiene nombres feos (`calcDesc`, `p`, `r`). No debería haber hallazgos.
3	`repo/tickets.ts` + `routes/tickets.ts`	El SQL se arma con `${id}`, pero la ruta valida que `id` sea un entero positivo. Prueba si el agente verifica antes de afirmar.

Resultado medido

Caso	Versión	Palabras	¿Contesta las 4 preguntas una por una?	Hallazgos	Con etiqueta de severidad
1	Antes	710	Sí	5	0

Caso	Versión	Palabras	¿Contesta las 4 preguntas una por una?	Hallazgos	Con etiqueta de severidad
1	Después	476	No	5	5
2	Antes	327	Sí	2 (inventados)	0
2	Después	**6**	No	**0**	—
3	Antes	564	Sí	5	0
3	Después	283	No	2 + 1 nota	2

Caso 2 – archivo sano (prueba los cambios 1 y 5)

Antes: respondió las cuatro preguntas (por ejemplo: "¿Qué se rompe con volumen? Nada. Es O(1)…") y aun así agregó dos hallazgos para no dejar la lista vacía:

> 1. Redondeo de punto flotante en montos (bajo) … `1.005 → 1`
> 2. Desborde con precios extremadamente grandes (muy bajo) … valores del orden de 1e306, imposibles en un precio real

Después:

> Sin hallazgos de seguridad o robustez.

Qué cambió: pasó de 327 palabras a 6. El primer hallazgo de "antes" no es de seguridad ni de robustez, y el segundo el propio agente lo describe como imposible en la práctica: estaba rellenando.

Caso 3 – validación en otro archivo (prueba los cambios 2, 3 y 4)

Antes: su hallazgo **número 1** fue:

> El SQL se arma interpolando el valor, y la seguridad depende de que valide otro archivo. Línea 6 …

Lo puso como el problema más grave del archivo, aunque en el mismo texto reconoce que "hoy no se puede explotar". También reportó la caché de planes de SQL Server y el control de acceso por ticket: cinco hallazgos sin severidad.

Después: confirmó en `routes/tickets.ts:7-10` que el `id` se valida, y **sacó la interpolación de la lista de hallazgos**. La dejó como nota aclarando cuándo se volvería explotable. Los dos hallazgos que quedaron son los que pueden tirar abajo el servicio, y uno está marcado **"A confirmar"** porque depende de la versión de Express, que no se puede ver en el código:

> **[Alto] repo/tickets.ts:4 — La consulta se ejecuta sin esperar a que el pool esté conectado**
> Escenario: … Si `pool.connect()` falla al arrancar … todas las consultas fallan, para todos los usuarios, hasta que se reinicie el proceso.
> **[Alto] repo/tickets.ts:4-6 — Los errores de la base no se capturan en ningún lado (A confirmar)**

Qué cambió: el orden. Con el revisor anterior, quien lea la revisión arreglaría primero algo que hoy no se puede explotar. Con el nuevo, arregla primero lo que puede dejar el servicio caído.

Caso 1 – la función del TP5 (prueba los cambios 3 y 4)

Antes: los cinco hallazgos son correctos, pero vienen en párrafos que mezclan varios problemas. Por ejemplo, el primero junta el corte del JSON de `FOR JSON` con el límite de 10 MB. Ninguno tiene severidad, y antes de los hallazgos repite casi todo en la sección de las cuatro preguntas.

Después: cada problema aparece separado y con severidad. El corte del JSON y el límite de 10 MB son dos hallazgos distintos (`[Alto] :28,38` y `[Alto] :10,28`). La exposición de datos por `SELECT *` pasó a ser el primero,

como **[Crítico — A confirmar]**, aclarando que no encontró quién llama a la función y qué habría que mirar (si la ruta exige autenticación). Es 33% más corto.

Lo que NO cambió (también es un hallazgo)

- **Qué problemas detecta.** En el caso 1 las dos versiones encontraron prácticamente lo mismo: corte del JSON, falta de timeout, tope de 10 MB, `-E` y `SELECT *`, y `DB_DATABASE` sin validar. Mis cambios **no hacen que el agente encuentre más cosas**: cambian el orden, la precisión y lo que se calla. Para detectar más habría que cambiar las preguntas, y eso no lo toqué.
- **Buscar quién llama a la función.** En el caso 3 la versión anterior **también** abrió `routes/tickets.ts` por su cuenta. El cambio 2 no hizo que lo leyera, sino que **usara lo que leyó** para decidir si era un hallazgo o no.

Limitaciones de la prueba

- Hice **una corrida por versión y caso**. Un modelo no responde siempre igual, así que una sola corrida muestra una tendencia, no una garantía. Para estar más seguro, habría que correr cada caso 3 a 5 veces y comparar.
- Las corridas se hicieron con Claude Code, pasándole al subagente las instrucciones del agente tal cual. No se hicieron en el entorno de clase, donde el archivo se carga por su `name`. El comportamiento debería ser equivalente, pero no es idéntico.